26F Stat TA Session W6

📢 Quiz 3 (`R` quiz) Information

- Time: **10/22 (Thu.) 15:30 - 16:10**
- **Quiz 3 has 40 mins only**
- We will have TA Session after the quiz!
- Coverage: TA session materials
- Multi-choice questions

0. Preparation for TA Session

- Download the sample code and take a look at the attached dataset: `ames` (housing prices in Ames, Iowa, from 2006 to 2010) click here
  - As usual, detailed descriptions of the dataset can be found at OpenIntro
- Remember to import the dataset into `RStudio`

1. Deal with Tricky data: Characters & Dates

- In W4, we get a taste of converting logical values into numbers using `as.numeric()` . But sometimes, such a simple manipulation trick cannot fulfill what we desire. Especially when it comes to characters (strings) and Date data
  - Recall the horrifying exercise you have done last week! 10 exams with exact 10 different names!

`paste()`

- As the name suggests, the function `paste()` save us the time to do copy-paste and paste strings together!

```
paste("test", 1:5, sep = "_")  # default = " "
[1] "test_1" "test_2" "test_3" "test_4" "test_5"

paste(c("A","B","C"), collapse = ", ")  # collapse combines strings into one
string
[1] "A, B, C"

paste("test", 1:5, sep = "^", collapse = "+")
[1] "test^1+test^2+test^3+test^4+test^5"
```

- `paste()` works pairwise across inputs (in the case you have inputs with different lengths)

```
paste(c("A", "B", "C", "D"), 1:2, sep = "_")
[1] "A_1" "B_2" "C_1" "D_2"
```

- You can also combine more than two vectors or strings!

```
paste("ID", 1:3, "Section", c("A", "B"), sep = "_")
[1] "ID_1_Section_A" "ID_2_Section_B" "ID_3_Section_A"
```

`as.Date()`

- Another tricky type of data we may encounter is **time information (date data)**
- In this case, we want date data to keep the numeric properties (e.g., additivity) instead of simply being a character

```
as.Date(string, format = "%m-%d-%Y")  # default: %Y-%m-%d

# example
birthday <- as.Date("2026-10-16")
class(birthday)
[1] "2026-10-16"
[1] "Date"

as.Date("10/16/2026", format = "%m/%d/%Y")
[1] "2026-10-16"
```

- As we have mentioned, we can do additions and subtractions to Date data

```
birthday + 7   # 7 days later
[1] "2026-10-23"

birthday - 365 # 365 days ago
[1] "2025-10-16"

christmas <- as.Date("2026-12-25")
christmas - birthday
# output: Time difference of 70 days
```

- Now you should be able to perform basic manipulation on Date data. Check `lubridate` for more advanced tricks on Date data

2. Data Visualization Revisited

- The best way to understand a dataset is to take a look at variable descriptions, take a look at the numbers, and draw figures!

- When Date data are available, we may want to observe the trend (e.g., does the price increases over time? is there dramatic changes of certain variables?)

What's in the dataset `ames`?

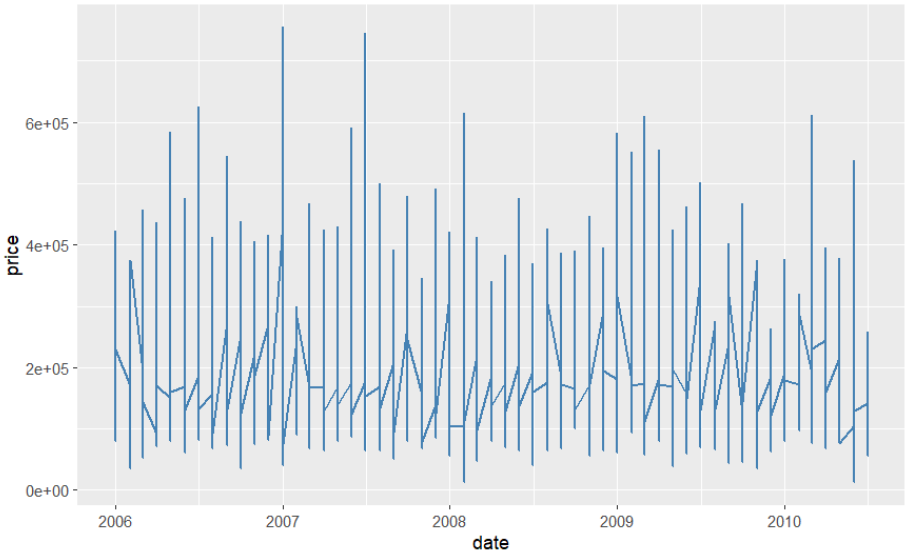
- `Yr.Sold` : Year Sold (YYYY)
- `Mo.Sold` : Month Sold (MM)
- `price` : Sale price in USD

Line Graph `geom_line()`

- Shows trends in data over time

```
# deal with Date data
housing$date <- as.Date(paste(housing$`Yr.Sold`,
                              housing$`Mo.Sold`,
                              "01",        # add %d for the format
                              sep = "-")   # we obtain yyyy-mm-01
                            )

ggplot(housing, aes(x = date, y = price)) +    # plot the time series data
  geom_line(linewidth = 0.7,
            color = "steelblue"
            )
```



What do we miss with the figure?

- Take a look at the data...
- Each vertical line shows the max and min within the date!

What's the better way to picture this?

- Show the **mean** housing price for each date

```
ggplot(housing, aes(x = date, y = price)) +
  geom_line(stat = "summary",         # take summary statistics
            fun = "mean",             # specify the used function
            linewidth = 0.7,
            color = "steelblue"
            )
```

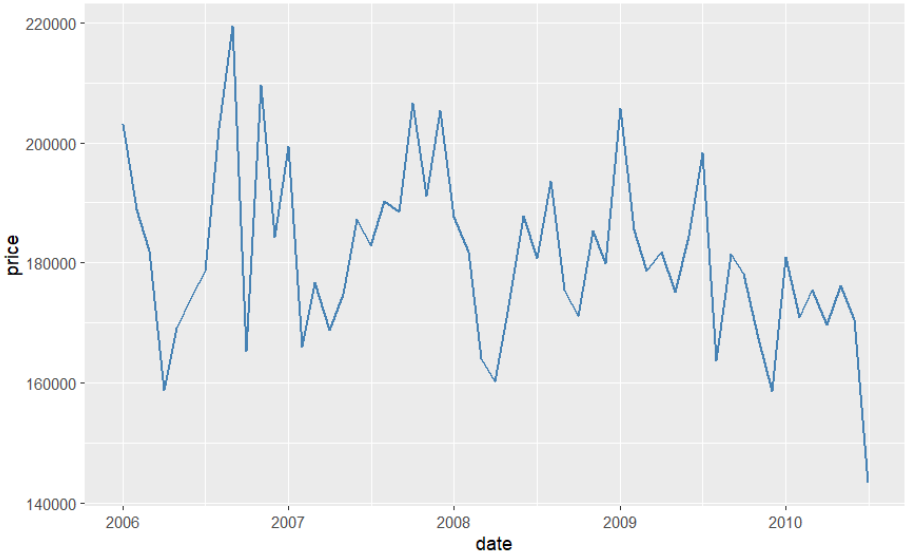


Figure: Average house sold price during 2006 to 2011

Exercises

W6 Exercises